

TASKFLOW

Gestor completo de tarefas

Relatório técnico da entrega final

Integrantes	Maria Eduarda e José Daniel
Curso	Análise e Desenvolvimento de Sistemas
Tecnologias	Next.js, TypeScript, Tailwind CSS e Firebase
Data	Junho de 2026

Links para preencher antes da entrega:

GitHub: [INSERIR LINK DO REPOSITÓRIO]

Vercel: [INSERIR LINK DA APLICAÇÃO PUBLICADA]

Sumário

1. Introdução e objetivos
2. Visão geral da arquitetura
3. Tecnologias utilizadas
4. Estrutura de pastas e arquivos
5. Modelagem de dados
6. Arquitetura do sistema
7. Implementação dos requisitos
8. Segurança e persistência
9. Acessibilidade digital
10. Testes, GitHub e deploy
11. Desafios técnicos e soluções
12. Conclusão

1. Introdução e objetivos

O TaskFlow é uma aplicação web para gerenciamento de tarefas pessoais. O projeto foi desenvolvido com o App Router do Next.js e utiliza o Firebase para autenticação e persistência em tempo real. A entrega final consolida as duas etapas parciais e adiciona as visualizações de Kanban e calendário, subtarefas, progresso, página de detalhes, recursos de acessibilidade e documentação de implantação.

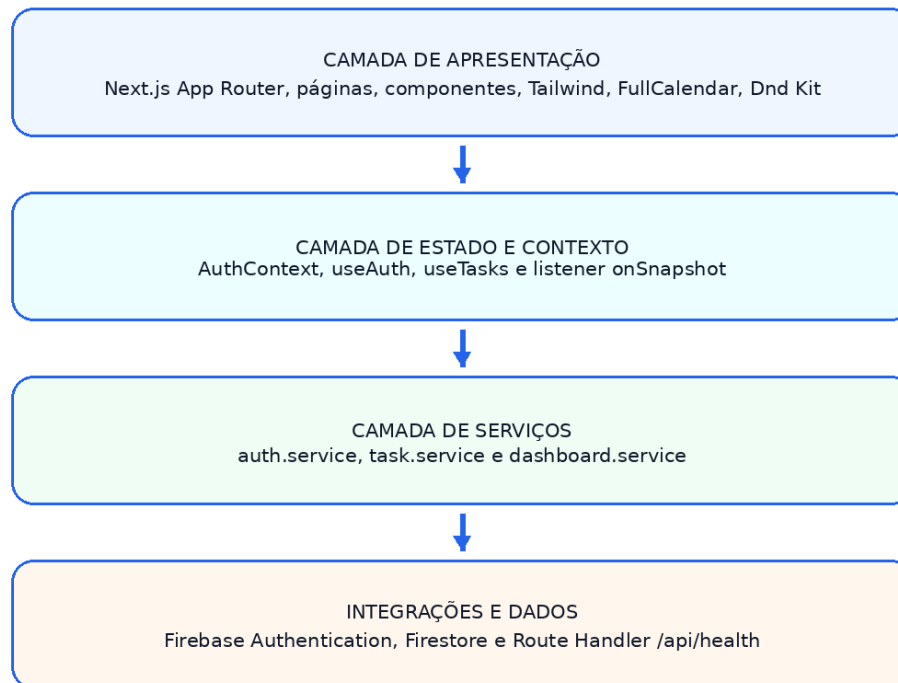
Os objetivos principais foram:

- Permitir cadastro e autenticação por e-mail, Google e GitHub.
- Garantir que cada usuário visualize apenas os próprios dados.
- Oferecer CRUD completo de tarefas e subtarefas.
- Apresentar métricas e gráficos atualizados em tempo real.
- Permitir organização por lista, quadro Kanban e calendário.
- Atender aos requisitos de responsividade, acessibilidade e boas práticas de arquitetura.

2. Visão geral da arquitetura

A aplicação foi organizada em camadas para separar interface, estado, regras de negócio e infraestrutura. As páginas do App Router consomem componentes reutilizáveis. Os hooks e contextos controlam autenticação e assinaturas em tempo real. Os serviços concentram as operações do Firebase, evitando chamadas diretas ao banco dentro das telas.

Arquitetura em Camadas - TaskFlow



Fluxo principal: Interface → Hooks/Contextos → Serviços → Firebase

Figura 1 - Diagrama em camadas da arquitetura do TaskFlow.

Fluxo de uma alteração de status no Kanban: o usuário arrasta um cartão; o Dnd Kit identifica a coluna de destino; o componente chama `updateTaskStatus`; o serviço atualiza o documento no Firestore; e o listener `onSnapshot` distribui o novo estado para Dashboard, Lista, Detalhes, Kanban e Calendário.

3. Tecnologias utilizadas

Camada	Tecnologia	Responsabilidade
Framework	Next.js 16 / React 19	App Router, páginas, componentes e Route Handler.
Linguagem	TypeScript	Tipagem forte para tarefas, subtarefas, usuários e serviços.
Estilo	Tailwind CSS 4	Layout responsivo, componentes e temas visuais.
Autenticação	Firebase Authentication	E-mail/senha, confirmação de e-mail, Google e GitHub.
Persistência	Cloud Firestore	Dados do usuário, tarefas, subtarefas e logs em tempo real.
Formulários	React Hook Form + Zod	Validação de cadastro, login e tarefas.
Dashboard	Tremor-style + Recharts	Dois gráficos: barras por status e rosca por prioridade.
Calendário	FullCalendar	Eventos por data de vencimento e modal de detalhes.
Kanban	Dnd Kit	Drag and drop entre as três colunas.
Interface	Framer Motion, Lucide, Sonner	Animações, ícones e notificações.
Temas	Next Themes	Tema claro, escuro e alto contraste.

4. Estrutura de pastas e arquivos

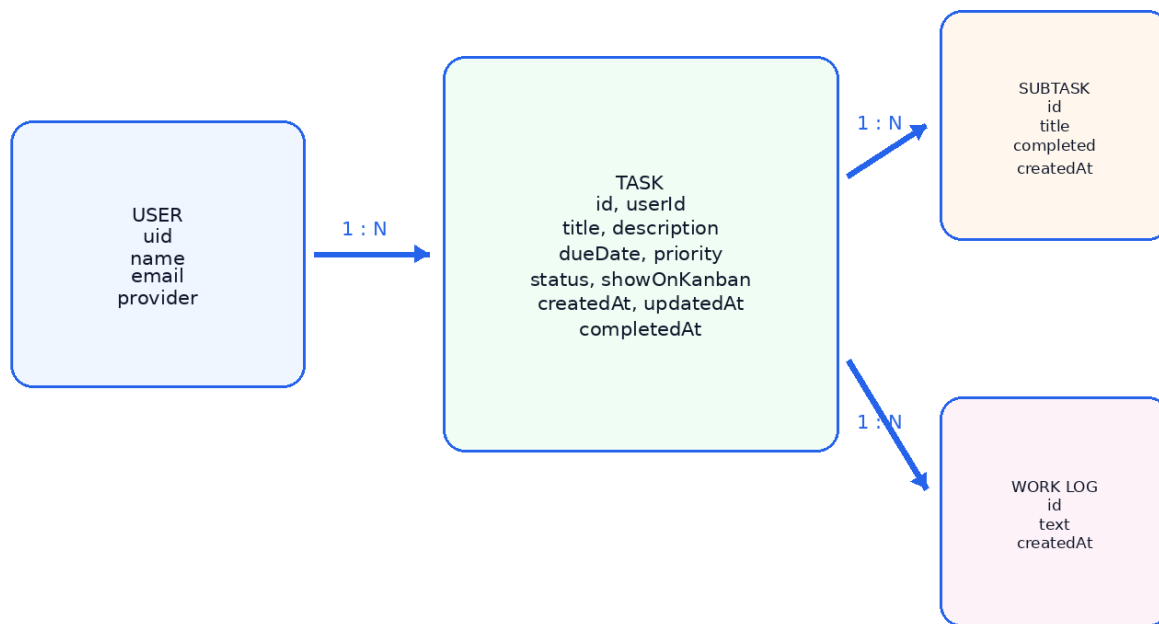
A estrutura segue uma divisão modular e escalável. Páginas ficam em `app`, componentes reutilizáveis em `components`, integrações em `lib`, regras de negócio em `services`, estados compartilhados em `contexts` e `hooks`, e modelos em `types`.

```
src/
├── app/
│   ├── api/health/route.ts
│   ├── calendar/page.tsx
│   ├── dashboard/page.tsx
│   ├── kanban/page.tsx
│   ├── login/page.tsx
│   ├── register/page.tsx
│   ├── tasks/
│   │   ├── [id]/page.tsx
│   │   └── page.tsx
│   ├── globals.css
│   ├── layout.tsx
│   └── page.tsx
├── components/
│   ├── tremor/
│   ├── ui/aceternity/
│   ├── AccessibilityPanel.tsx
│   ├── DashboardCharts.tsx
│   ├── KanbanBoard.tsx
│   ├── SubtaskManager.tsx
│   ├── TaskForm.tsx
│   └── VLibras.tsx
├── contexts/AuthContext.tsx
├── hooks/useAuth.ts e useTasks.ts
├── lib/firebase.ts e utils.ts
├── services/auth.service.ts, task.service.ts e dashboard.service.ts
├── types/task.ts e user.ts
└── proxy.ts
```

5. Modelagem de dados

O documento principal é Task. Subtask e WorkLog são armazenados como listas dentro da tarefa, porque sempre são exibidos e atualizados no contexto de uma tarefa específica. O campo `userId` estabelece a propriedade do documento e é validado pelas regras do Firestore.

Modelagem de Dados



Cada tarefa pertence a um usuário e contém listas embutidas de subtarefas e registros de trabalho.

Figura 2 - Relacionamento lógico entre usuário, tarefa, subtarefa e log.

5.1 Interfaces TypeScript

```

export type Subtask = {
  id: string;
  title: string;
  completed: boolean;
  createdAt: string;
};

export type WorkLog = {
  id: string;
  text: string;
  createdAt: string;
};

export type Task = {
  id: string;
  userId: string;
  title: string;
  description: string;
  dueDate: string;
  priority: "baixa" | "media" | "alta";
  status: "a-fazer" | "fazendo" | "concluido";
  showOnKanban: boolean;
  subtasks: Subtask[];
  workLogs: WorkLog[];
  createdAt: string;
  updatedAt: string;
  completedAt: string | null;
};
  
```

6. Arquitetura do sistema

6.1 Camada de apresentação

Inclui landing page, formulários, dashboard, lista de tarefas, página de detalhes, quadro Kanban e calendário. Componentes pequenos e reutilizáveis foram empregados para barra de progresso, cards, modais, acessibilidade e gráficos.

6.2 Contextos e hooks

O AuthContext observa o estado do Firebase Authentication. O hook useTasks cria uma assinatura `onSnapshot` filtrada pelo UID, permitindo que alterações feitas em qualquer página apareçam imediatamente nas demais.

6.3 Serviços

A camada de serviços implementa cadastro, login, reautenticação, exclusão de conta, CRUD de tarefas, atualização de status, visibilidade no Kanban, subtarefas e logs. O dashboard.service calcula métricas sem duplicar regras dentro da interface.

6.4 Infraestrutura

O Firebase Authentication mantém as identidades. O Firestore armazena perfis e tarefas. O Route Handler `/api/health` demonstra o uso de API do App Router. O Proxy do Next.js atua como primeira barreira de navegação, enquanto a autorização real dos dados é garantida pelas regras do Firestore.

7. Implementação dos requisitos

Requisito	Implementação
Landing page	Menu responsivo, footer institucional, call to action, mockup, ícone de login e animações no estilo Aceternity.
Cadastro	Nome, e-mail, senha forte, confirmação, campos obrigatórios e validação com zodResolver.
Login	E-mail/senha com confirmação de e-mail, Google e GitHub por Firebase Authentication.
Dashboard	Métricas de pendentes, concluídas na semana e vencidas; dois gráficos de dados.
CRUD	Criação, leitura, edição e exclusão com título, descrição, vencimento, prioridade e status.
Subtarefas	Adicionar, remover e marcar; barra calculada automaticamente.
Calendário	FullCalendar em página dedicada; evento automático; clique abre modal.
Kanban	Colunas A Fazer, Fazendo e Concluído; Dnd Kit; atualização no Firestore.
Lista para Kanban	Campo showOnKanban e botão para adicionar ou remover uma tarefa do quadro.
Detalhes	View por ID, edição completa, subtarefas, status e log de trabalho.
Acessibilidade	VLibras, temas, escala de texto, redução de movimento, links sublinhados, foco e skip link.

8. Segurança e persistência

Cada tarefa recebe o UID do usuário autenticado. As consultas usam `where("userId", "==", uid)`. As regras do Firestore comparam `request.auth.uid` com o proprietário do documento, impedindo leitura ou alteração de tarefas de terceiros.

```
match /tasks/{taskId} {
  allow create: if request.auth != null
    && request.auth.uid == request.resource.data.userId;
  allow read: if request.auth != null
    && request.auth.uid == resource.data.userId;
  allow update: if request.auth != null
    && request.auth.uid == resource.data.userId
    && request.resource.data.userId == resource.data.userId;
  allow delete: if request.auth != null
    && request.auth.uid == resource.data.userId;
}
```

O arquivo `.env.local` contém somente configurações públicas do aplicativo Web do Firebase, mas permanece fora do Git por organização. As operações sensíveis dependem da sessão autenticada e das regras do banco. Para excluir a conta, usuários de e-mail/senha precisam informar a senha atual; provedores sociais realizam nova autenticação por popup.

9. Acessibilidade digital

Foram implementados recursos além do mínimo de cinco solicitado:

1. VLibras integrado globalmente para tradução em Libras.
2. Temas claro, escuro e alto contraste.
3. Aumento e redução do texto entre 100%, 115% e 130%.
4. Redução de animações para usuários sensíveis a movimento.
5. Opção de sublinhar todos os links.
6. Foco visível em elementos interativos.
7. Link “Pular para o conteúdo principal”.
8. Rótulos, mensagens de erro com role alert e atributos ARIA.
9. Operação por teclado no Kanban com instruções de leitor de tela.

10. Testes, GitHub e deploy

10.1 Verificações técnicas

```
npm install
npm run lint
npm run typecheck
npm run build
```

O projeto inclui também o comando `npm run check`, que executa lint, verificação de tipos e build em sequência.

10.2 Repositório GitHub

O README contém instruções de cópia, instalação, configuração do Firebase, regras do Firestore, execução local, testes e implantação. A contribuição da Maria deve ser realizada em branch própria e integrada à main por Pull Request, preservando a autoria do commit.

Link do repositório: [INSERIR LINK DO GITHUB]

10.3 Publicação na Vercel

10. Importar o repositório após o merge na main.
11. Adicionar todas as variáveis NEXT_PUBLIC_FIREBASE_* em Environment Variables.
12. Fazer o deploy.
13. Adicionar o domínio da Vercel aos Authorized domains do Firebase Authentication.
14. Testar autenticação social, Firestore, Kanban e calendário em produção.

Link publicado: [INSERIR LINK DA VERCEL]

11. Desafios técnicos e soluções

11.1 Estado complexo no drag and drop

Desafio: manter o quadro responsivo e sincronizado com o Firestore ao mover um cartão. Solução: o Dnd Kit fornece o ID da tarefa e da coluna de destino; o serviço persiste o novo status; e o listener em tempo real atualiza todas as telas sem duplicar estado local.

11.2 Calendário com muitas tarefas

Desafio: converter documentos em eventos e localizar rapidamente o item clicado. Solução: somente tarefas com data são transformadas; o ID é preservado em `extendedProps`; o FullCalendar limita eventos visíveis por dia com `dayMaxEvents`. Em uma evolução com grande volume, seria possível filtrar por intervalo de datas e paginar consultas.

11.3 Consistência do progresso

Desafio: impedir divergência entre subtarefas e porcentagem. Solução: o progresso não é salvo como campo independente. Ele é recalculado a partir da quantidade de subtarefas concluídas, eliminando dados duplicados.

11.4 Autorização em aplicação cliente

Desafio: um redirecionamento no navegador não é suficiente para proteger o banco. Solução: o Proxy melhora a experiência de navegação, mas as regras do Firestore são a fonte real de autorização e validam o UID em cada operação.

12. Conclusão

A entrega final do TaskFlow atende aos requisitos funcionais e tecnológicos propostos. O sistema possui autenticação completa, persistência por usuário, CRUD, subtarefas, progresso, gráficos, calendário, Kanban, detalhes, log de trabalho e recursos de acessibilidade. A estrutura modular facilita manutenção e expansão, enquanto o README e este relatório documentam a execução local, o repositório e a publicação.

Antes da submissão, devem ser preenchidos somente os dois campos de link do GitHub e da Vercel, pois esses endereços dependem das contas dos integrantes e não podem ser gerados automaticamente no relatório.